In [ ]:
```
import marimo as mo
```

Multiplicação de inteiros

Modelo do CFA da multiplicação como SFOTS em Z3, com BitVec de tamanho n, e provas por k-indução.

In [ ]:
```
# o marimo não aceita "from z3 import *", por isso importa-se só o que o código usa
from z3 import Int, BitVec, And, Or, Not, If, Implies, ULT, UGE, LShR, BV2Int, ZeroExt, ForAll, Solver, sat, unsat, unknown, is_true, is_false
```

Constantes

`n` é o número de bits e os nodos do CFA são numerados, para o `pc` poder ser um `Int`. O modelo funciona para qualquer n par, mas usa-se $n = 8$ porque a prova do ponto 2 multiplica BitVec e fica lenta depressa (com $n = 16$ não acaba em 5 minutos).

In [ ]:
```
n = 8   # número de bits
assert n % 2 == 0, 'n tem de ser par, por causa do 2^(n/2) do ponto 3'

INIT, SKIP, LEFT, RIGHT, STOP, ERROR = 0, 1, 2, 3, 4, 5
NOMES = ['init', 'skip', 'left', 'right', 'stop', 'error']
```

Ponto 1 — o modelo

- **Variáveis:** `pc` (nodo atual, `Int`) e x, y, z, a, b (BitVec de n bits).
- **Estado inicial** (`init`): $pc = init$, com a e b quaisquer e x, y, z livres.
- **Relação de transição** (`trans`): um ramo por aresta do CFA, mais os laços em stop e error; a e b nunca mudam.
- **Estado de erro** (`error`): $pc = error$, onde se vai parar quando há overflow.

In [ ]:
```
def declare(i):
    s = {}
    s['pc'] = Int('pc' + str(i))
    s['x'] = BitVec('x' + str(i), n)
    s['y'] = BitVec('y' + str(i), n)
    s['z'] = BitVec('z' + str(i), n)
    s['a'] = BitVec('a' + str(i), n)
    s['b'] = BitVec('b' + str(i), n)
    return s
```

In [ ]:
```
def init(s):
    return s['pc'] == INIT
```

```
def error(s):
    return s['pc'] == ERROR
```

In [ ]:
```
def par(v):
    return (v & 1) == 0


def overflow_dobro(x):
    # 2*x não cabe em n bits quando x >= 2^(n-1)
    return UGE(x, 2**(n-1))


def overflow_soma(z, x):
    # se a soma deu a volta, o resultado fica mais pequeno do que z
    return ULT(z + x, z)


def iguais(s, p):
    # x, y e z não mudam
    return And(p['x'] == s['x'], p['y'] == s['y'], p['z'] == s['z'])
```

In [ ]:
```
def trans(s, p):
    x = s['x']
    y = s['y']
    z = s['z']

    # a e b nunca mudam
    ab_iguais = And(p['a'] == s['a'],
                    p['b'] == s['b'])

    init_skip = And(s['pc'] == INIT,
                    p['pc'] == SKIP,
                    p['x'] == s['a'],
                    p['y'] == s['b'],
                    p['z'] == 0)

    skip_left = And(s['pc'] == SKIP,
                    y != 0,
                    par(y),
                    p['pc'] == LEFT,
                    iguais(s, p))

    skip_right = And(s['pc'] == SKIP,
                    y != 0, Not(par(y)),
                    p['pc'] == RIGHT,
                    iguais(s, p))

    skip_stop = And(s['pc'] == SKIP,
                    y == 0,
                    p['pc'] == STOP,
                    iguais(s, p))

    left_skip = And(s['pc'] == LEFT,
                    Not(overflow_dobro(x)),
```

```
                    p['pc'] == SKIP,
                    p['x'] == (x << 1),
                    p['y'] == LShR(y, 1),
                    p['z'] == z)

    left_error = And(s['pc'] == LEFT,
                    overflow_dobro(x),
                    p['pc'] == ERROR,
                    iguais(s, p))

    right_skip = And(s['pc'] == RIGHT, Not(overflow_soma(z, x)),
                    p['pc'] == SKIP,
                    p['x'] == x, p['y'] == y - 1,
                    p['z'] == z + x)

    right_error = And(s['pc'] == RIGHT, overflow_soma(z, x),
                    p['pc'] == ERROR, iguais(s, p))

    stop_stop = And(s['pc'] == STOP,
                    p['pc'] == STOP, iguais(s, p))

    error_error = And(s['pc'] == ERROR,
                    p['pc'] == ERROR, iguais(s, p))

    return And(ab_iguais, Or(init_skip, skip_left, skip_right, skip_stop,
                             left_skip, left_error, right_skip,
right_error,
                             stop_stop, error_error))
```

Para a elaboração do código acima foi usado o recurso à LLM Claude.

Link para a conversa: https://claude.ai/share/a06cc141-81a6-4cc3-9156-89cb2303ebc2

Verificações do modelo

Confirma-se que `trans` é determinística e total. As duas funções procuram um contraexemplo, por isso o resultado esperado é `unsat`.

In [ ]:
```
def diferentes(p, q):
    return Or(p['pc'] != q['pc'], p['x'] != q['x'], p['y'] != q['y'],
              p['z'] != q['z'], p['a'] != q['a'], p['b'] != q['b'])


def deterministico():
    e = declare(0)
    p1 = declare(1)
    p2 = declare(2)
    s = Solver()
    s.add(trans(e, p1))
    s.add(trans(e, p2))
    s.add(diferentes(p1, p2))
    return s.check()
```

```
def total():
    # procura um estado com pc válido sem nenhum sucessor (deve dar unsat)
    e = declare(0)
    p = declare(1)
    s = Solver()
    s.add(e['pc'] >= INIT, e['pc'] <= ERROR)   # os nodos são 0..5
    s.add(ForAll([p['pc'], p['x'], p['y'], p['z'], p['a'], p['b']],
Not(trans(e, p))))
    return s.check()
```

In [ ]:
```
print('Determinismo:', deterministico())
print('Totalidade:', total())
```

```
Determinismo: unsat
Totalidade: unsat
```

Simulação e testes

`testar` confirma, para vários pares de a e b, que o programa acaba em stop com $z = a \cdot b$ ou, se o produto não couber em n bits, em error. Os três traços mostram o maior produto que cabe e os dois tipos de overflow (no dobro e na soma).

In [ ]:
```
def ler_traco(m, traco, inv=None):
    estados = []
    for s in traco:
        e = {}
        for v in ['pc', 'x', 'y', 'z', 'a', 'b']:
            e[v] = m.eval(s[v], model_completion=True).as_long()
        if inv is not None:
            e['inv'] = is_true(m.eval(inv(s), model_completion=True))
        estados.append(e)
    return estados


def imprime_traco(estados):
    for i in range(len(estados)):
        e = estados[i]
        linha = str(i) + '  ' + NOMES[e['pc']]
        for v in ['x', 'y', 'z', 'a', 'b']:
            linha = linha + '  ' + v + '=' + str(e[v])
            if e['pc'] == INIT and v in ['x', 'y', 'z']:
                linha = linha + ' (livre)'
        if 'inv' in e:
            linha = linha + '  inv=' + str(e['inv'])
        print(linha)
        # a partir de stop ou error o estado já não muda
        if e['pc'] == STOP or e['pc'] == ERROR:
            break


def simular(a_val, b_val):
    assert 0 <= a_val < 2**n, 'a não cabe em n bits'
    assert 0 <= b_val < 2**n, 'b não cabe em n bits'
    passos = 4 * n   # nenhuma execução passa das 4n transições
```

```
    traco = []
    for i in range(passos + 1):
        traco.append(declare(i))

    s = Solver()
    s.add(init(traco[0]))
    s.add(traco[0]['a'] == a_val)
    s.add(traco[0]['b'] == b_val)

    for i in range(passos):
        s.add(trans(traco[i], traco[i+1]))

    r = s.check()

    if r == sat:
        return ler_traco(s.model(), traco)
    elif r == unsat:
        raise Exception('unsat: não há traço com ' + str(passos) + '
passos')
    else:
        raise Exception('unknown: o Z3 não conseguiu decidir')
```

In [ ]:
```
def correto(a_val, b_val, final):
    if a_val * b_val < 2**n:
        return final['pc'] == STOP and final['z'] == a_val * b_val
    else:
        return final['pc'] == ERROR


def valores_teste():
    if n <= 4:
        return list(range(2**n))
    raiz = 2**(n // 2)
    return [0, 1, 2, 3, raiz - 1, raiz, raiz + 1, 2**(n-1) - 1, 2**(n-1),
2**n - 1]


def testar(valores):
    falhas = []
    for a_val in valores:
        for b_val in valores:
            estados = simular(a_val, b_val)
            if not correto(a_val, b_val, estados[-1]):
                falhas.append((a_val, b_val))
    return falhas
```

In [ ]:
```
valores = valores_teste()
falhas = testar(valores)
print('n =', n, '|', len(valores)**2, 'pares testados |', len(falhas),
'falhas:', falhas)
```

```
n = 8 | 100 pares testados | 0 falhas: []
```

In [ ]:
```
imprime_traco(simular(2**(n//2) - 1, 2**(n//2) + 1))
```

```
0  init  x=0 (livre)  y=0 (livre)  z=0 (livre)  a=15  b=17
1  skip  x=15  y=17  z=0  a=15  b=17
2  right  x=15  y=17  z=0  a=15  b=17
3  skip  x=15  y=16  z=15  a=15  b=17
4  left  x=15  y=16  z=15  a=15  b=17
5  skip  x=30  y=8  z=15  a=15  b=17
6  left  x=30  y=8  z=15  a=15  b=17
7  skip  x=60  y=4  z=15  a=15  b=17
8  left  x=60  y=4  z=15  a=15  b=17
9  skip  x=120  y=2  z=15  a=15  b=17
10  left  x=120  y=2  z=15  a=15  b=17
11  skip  x=240  y=1  z=15  a=15  b=17
12  right  x=240  y=1  z=15  a=15  b=17
13  skip  x=240  y=0  z=255  a=15  b=17
14  stop  x=240  y=0  z=255  a=15  b=17
```

In [ ]:
```
imprime_traco(simular(2**(n//2), 2**(n//2)))
```

```
0  init  x=0 (livre)  y=0 (livre)  z=0 (livre)  a=16  b=16
1  skip  x=16  y=16  z=0  a=16  b=16
2  left  x=16  y=16  z=0  a=16  b=16
3  skip  x=32  y=8  z=0  a=16  b=16
4  left  x=32  y=8  z=0  a=16  b=16
5  skip  x=64  y=4  z=0  a=16  b=16
6  left  x=64  y=4  z=0  a=16  b=16
7  skip  x=128  y=2  z=0  a=16  b=16
8  left  x=128  y=2  z=0  a=16  b=16
9  error  x=128  y=2  z=0  a=16  b=16
```

In [ ]:
```
imprime_traco(simular(2**(n-1) - 1, 3))
```

```
0  init  x=0 (livre)  y=0 (livre)  z=0 (livre)  a=127  b=3
1  skip  x=127  y=3  z=0  a=127  b=3
2  right  x=127  y=3  z=0  a=127  b=3
3  skip  x=127  y=2  z=127  a=127  b=3
4  left  x=127  y=2  z=127  a=127  b=3
5  skip  x=254  y=1  z=127  a=127  b=3
6  right  x=254  y=1  z=127  a=127  b=3
7  error  x=254  y=1  z=127  a=127  b=3
```

k-indução

`kinduction_always` testa o caso base e depois o passo: dá FALSA se o caso base falha, INCONCLUSIVA se só o passo falha e PROVADA se ambos passam.

In [ ]:
```
def verifica(s):
    r = s.check()
    if r == unknown:
        raise RuntimeError('o Z3 respondeu unknown: ' + s.reason_unknown())
    return r


def kinduction_always(declare, init, trans, inv, k):

    if k < 1:
```

```
        raise ValueError('k tem de ser pelo menos 1')

    traco = []
    for i in range(k + 1):
        traco.append(declare(i))

    s = Solver()
    s.add(init(traco[0]))
    for i in range(k - 1):
        s.add(trans(traco[i], traco[i+1]))
    falhas = []
    for i in range(k):
        falhas.append(Not(inv(traco[i])))
    s.add(Or(falhas))
    if verifica(s) == sat:
        m = s.model()
        fim = k - 1

        for i in range(k):
            if is_false(m.eval(inv(traco[i]), model_completion=True)):
                fim = i
                break
        print(f'k = {k}: FALSA. Execução real, a partir de init, que viola
inv:')

        imprime_traco(ler_traco(m, traco[:fim + 1], inv))

        return False

    # o passo usa o mesmo solver, sem as asserções do caso base (como o
reset_assertions da ficha 8)
    s.reset()
    for i in range(k):
        s.add(inv(traco[i]))
        s.add(trans(traco[i], traco[i+1]))
    s.add(Not(inv(traco[k])))
    if verifica(s) == sat:
        print(f'k = {k}: INCONCLUSIVA. O passo falha; estes estados podem
não ser alcançáveis:')
        imprime_traco(ler_traco(s.model(), traco, inv))
        return False

    print(f'k = {k}: PROVADA. inv é invariante.')
    return True
```

Para a realização da k-indução e das funções de simulação e de testes foi usado o recurso à LLM Claude.

Link para a conversa: https://claude.ai/share/788d45be-9ca5-4cc2-b450-8e8ddd361886

Ponto 2 — $x \cdot y + z = a \cdot b$ é invariante?

Não em init, onde x, y e z estão livres, por isso `prop_mult` dá FALSA. Fora de init é (`inv_mult`): com $k = 1$ dá INCONCLUSIVA por causa de um left com y ímpar, que não é alcançável, e com $k = 2$ dá PROVADA, porque o estado anterior garante que y é par.

In [ ]:
```
def prop_mult(s):
    return s['x'] * s['y'] + s['z'] == s['a'] * s['b']


def inv_mult(s):
    return Or(s['pc'] == INIT, prop_mult(s))
```

In [ ]:
```
kinduction_always(declare, init, trans, prop_mult, 1)
```

```
 k = 1: FALSA. Execução real, a partir de init, que viola inv:
 0  init  x=0 (livre)  y=1 (livre)  z=255 (livre)  a=0  b=1  inv=False
False
```

In [ ]:
```
kinduction_always(declare, init, trans, inv_mult, 1)
```

```
 k = 1: INCONCLUSIVA. O passo falha; estes estados podem não ser alcançáveis:
 0  left  x=117  y=1  z=255  a=52  b=81  inv=True
 1  skip  x=234  y=0  z=255  a=52  b=81  inv=False
False
```

In [ ]:
```
kinduction_always(declare, init, trans, inv_mult, 2)
```

```
 k = 2: PROVADA. inv é invariante.
True
```

Para a realização do código do ponto 2 foi usado o recurso à LLM Claude.

Link para a conversa: https://claude.ai/share/f8f34cb3-97c1-44e8-9ba7-df3c4ee6fade

Ponto 3 — segurança com $a, b < 2^{n/2}$

`inv_seguro` diz que nunca se chega a error. Com a e b quaisquer dá FALSA; com $a, b < 2^{n/2}$ dá PROVADA com $k = 4n$, o máximo de transições até parar. Em alternativa, `inv_forte` junta-lhe os limites de a e b e $x \cdot y + z = a \cdot b$ em $2n$ bits (onde não dá a volta): implica `inv_seguro` e dá PROVADA com $k = 2$.

In [ ]:
```
def init_limitado(s):
    return And(init(s), ULT(s['a'], 2**(n // 2)), ULT(s['b'], 2**(n // 2)))


def inv_seguro(s):
    return Not(error(s))


def em_2n_bits(v):
```

```
    return ZeroExt(n, v)


def inv_forte(s):
    x = em_2n_bits(s['x'])
    y = em_2n_bits(s['y'])
    z = em_2n_bits(s['z'])
    a = em_2n_bits(s['a'])
    b = em_2n_bits(s['b'])
    return And(inv_seguro(s),
               ULT(s['a'], 2**(n // 2)), ULT(s['b'], 2**(n // 2)),
               Or(s['pc'] == INIT, x * y + z == a * b))
```

In [ ]:
```
kinduction_always(declare, init, trans, inv_seguro, 4 * n)
```

```
k = 32: FALSA. Execução real, a partir de init, que viola inv:
0  init  x=0 (livre)  y=0 (livre)  z=0 (livre)  a=2  b=136  inv=True
1  skip  x=2  y=136  z=0  a=2  b=136  inv=True
2  left  x=2  y=136  z=0  a=2  b=136  inv=True
3  skip  x=4  y=68  z=0  a=2  b=136  inv=True
4  left  x=4  y=68  z=0  a=2  b=136  inv=True
5  skip  x=8  y=34  z=0  a=2  b=136  inv=True
6  left  x=8  y=34  z=0  a=2  b=136  inv=True
7  skip  x=16  y=17  z=0  a=2  b=136  inv=True
8  right  x=16  y=17  z=0  a=2  b=136  inv=True
9  skip  x=16  y=16  z=16  a=2  b=136  inv=True
10  left  x=16  y=16  z=16  a=2  b=136  inv=True
11  skip  x=32  y=8  z=16  a=2  b=136  inv=True
12  left  x=32  y=8  z=16  a=2  b=136  inv=True
13  skip  x=64  y=4  z=16  a=2  b=136  inv=True
14  left  x=64  y=4  z=16  a=2  b=136  inv=True
15  skip  x=128  y=2  z=16  a=2  b=136  inv=True
16  left  x=128  y=2  z=16  a=2  b=136  inv=True
17  error  x=128  y=2  z=16  a=2  b=136  inv=False
```

False

In [ ]:
```
kinduction_always(declare, init_limitado, trans, inv_seguro, 4 * n)
```

```
k = 32: PROVADA. inv é invariante.
```

True

In [ ]:
```
def mais_forte(forte, fraco):
    # procura um estado em que forte é verdade e fraco é falso
    e = declare(0)
    s = Solver()
    s.add(forte(e))
    s.add(Not(fraco(e)))
    return s.check()   # unsat: forte implica fraco
```

In [ ]:
```
print('inv_forte -> inv_seguro:', mais_forte(inv_forte, inv_seguro))
```

```
inv_forte -> inv_seguro: unsat
```

In [ ]:
```
kinduction_always(declare, init_limitado, trans, inv_forte, 2)
```

```
k = 2: PROVADA. inv é invariante.
```

True

Para a realização do código do ponto 3 foi usado o recurso à LLM Claude.

Link para a conversa: https://claude.ai/share/6a997010-26f6-4b43-9b9d-3c9501ea3c40

Ponto 4 — terminação

A propriedade de animação é F G $(pc = stop \vee pc = error)$: para quaisquer a e b, a execução chega a stop ou error e, pelos laços, fica lá.

Como na ficha 8, usa-se um variante V, calculado nos inteiros com `BV2Int` para não dar a volta: 2^{n+1} em init, $2y + 1$ em skip, $2y$ em left e right e 0 em stop e error. Provam-se com $k = 1$ três propriedades de segurança:

- `variante_nao_negativo`: $V \geq 0$;
- `decrescente`: em cada transição, V desce ou passa a 0;
- `variante_zero`: se $V = 0$, o programa está em stop ou error.

Como V é natural, não pode descer para sempre: chega a 0, não sai de lá e, pela última propriedade, o programa terminou.

In [ ]:
```python
def terminou(s):
    return Or(s['pc'] == STOP, s['pc'] == ERROR)


def variante(s):
    Y = BV2Int(s['y'])
    return If(s['pc'] == INIT, 2 * 2**n,
              If(s['pc'] == SKIP, 2 * Y + 1,
                 If(Or(s['pc'] == LEFT, s['pc'] == RIGHT), 2 * Y,
                    0)))


def variante_nao_negativo(s):
    return variante(s) >= 0


def decrescente(s):
    # p é o estado seguinte; declare(-1) dá nomes (pc-1, x-1, ...) que não
aparecem no traço
    p = declare(-1)
    return ForAll([p['pc'], p['x'], p['y'], p['z'], p['a'], p['b']],
                  Implies(trans(s, p),
                          Or(variante(p) < variante(s), variante(p) == 0)))


def variante_zero(s):
    return Implies(variante(s) == 0, terminou(s))
```

In [ ]:
```
kinduction_always(declare, init, trans, variante_nao_negativo, 1)
```

```
 k = 1: PROVADA. inv é invariante.
```
True

In [ ]:
```
kinduction_always(declare, init, trans, decrescente, 1)
```

```
 k = 1: PROVADA. inv é invariante.
```
True

In [ ]:
```
kinduction_always(declare, init, trans, variante_zero, 1)
```

```
 k = 1: PROVADA. inv é invariante.
```
True

Para a realização do código do ponto 4 foi usado o recurso à LLM Claude.

Link para a conversa: https://claude.ai/share/6a997010-26f6-4b43-9b9d-3c9501ea3c40